Proiectarea Algoritmilor

Curs 3 – Programare dinamică



Bibliografie

- Cormen – Introducere în Algoritmi - Programare dinamică (cap. 16)
- Giumale – Introducere în Analiza Algoritmilor cap 4.5
- http://www.cse.ust.hk/~dekai/271/notes/L12/L12.pdf
- Diverse moduri de implementare a șirului lui Fibonacci, Cel mai lung subșir comun, Înmulțirea matricelor : https://colab.research.google.com/gist/johnthebrave/897ee3fdc3c1e0884a39fef913872a36/dynamic-programming-applications.ipynb#scrollTo=yq08PwdoxP20
- Arbori optimi de căutare: https://gist.github.com/johnthebrave/1c8a9b7de6a6edd7b8d756fa1135a635

Programare dinamică

- Programare dinamică
  - Descriere generală
  - Algoritm generic
  - Caracteristici
- Exemplificare: Șirul lui Fibonacci
- Exemplificare: Problema discretă a rucsacului
- Exemplificare: Înmulțirea matricilor
- Exemplificare: Arbori optimi la căutare (AOC)
  - Definiții
  - Construcția AOC



Programare dinamică

- Descriere generală
  - Soluții optime sau de combinatorică construite iterativ asamblând soluții optime ale unor probleme similare de dimensiuni mai mici.

- Algoritmi “clasici”
  - Înmulţirea unui şir de matrici
  - AOC
  - Algoritmul Floyd-Warshall care determină drumurile de cost minim dintre toate perechile de noduri ale unui graf.
  - Numere catalane
  - Viterbi

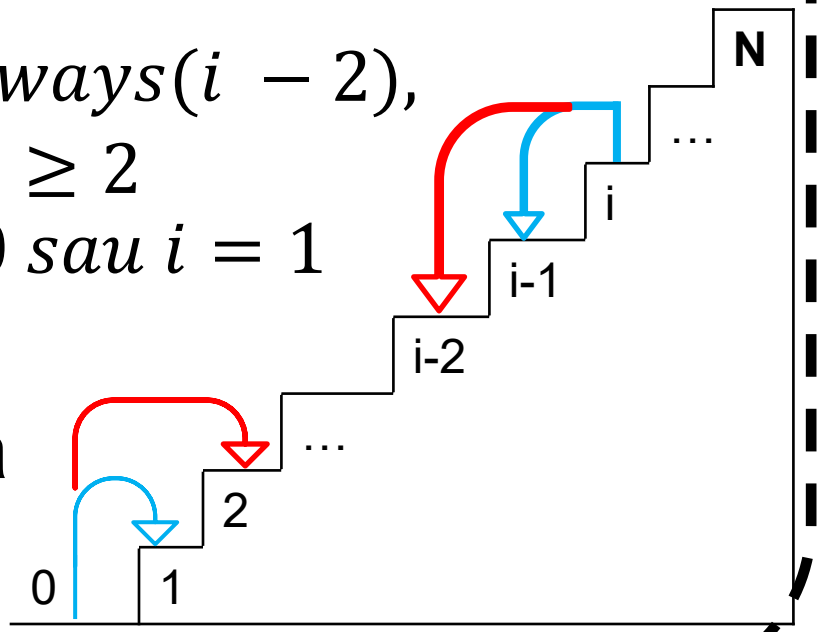
Algoritm generic

- Programare dinamică (crit_optim, problema)
  - // fie $problema_0$ $problema_1$ … $problema_n$ astfel încât
  - // $problema_n$ = problema; $problema_i$ mai simplă decât $problema_{i+1}$
  - 1. Sol = soluții_inițiale(crit_optim, $problema_0$);
  - 2. **Pentru** *i* **de la** 1 **la** *n* // construcție soluții pentru
    // $problema_i$ folosind soluțiile problemelor precedente
  - 3. Sol_i = calcul_soluții(Sol, crit_optim, $Problema_i$);
    // determin soluția $problemei_i$
  - 4. Sol = Sol U Sol_i; // memoizare
    // noile soluții se adaugă pentru a fi refolosite pe viitor
  - 5. s = soluție_pentru_$problema_n$(Sol);
    // selecție / construcție soluție finală
  - 6. **Întoarce** s;

Exemplu: Urcatul scărilor (1)

- În câte feluri se poate urca o scară ce are n trepte, având în vedere că la fiecare pas se poate urca fie 1 treaptă, fie 2.

$$ways(i) = \begin{cases} ways(i-1) + ways(i-2), \\ dacă\ \ i \geq 2 \\ 1, dacă\ i = 0\ sau\ i = 1 \end{cases}$$



- Transformăm recurența într-un tablou de valori (memoizare)

Exemplu: Urcatul scărilor (2)

- Abordare bottom-up:
  - ways[0] = ways[1] = 1
  - FOR i = 2 .. n:
    - ways[i] = ways[i-1] + ways[i-2]
  - RETURN ways[n]

- Complexitate: $\Theta(n)$

- Se poate îmbunătăți cu exponențiere matrici și atunci e $\Theta(\log(n))$



Caracteristici

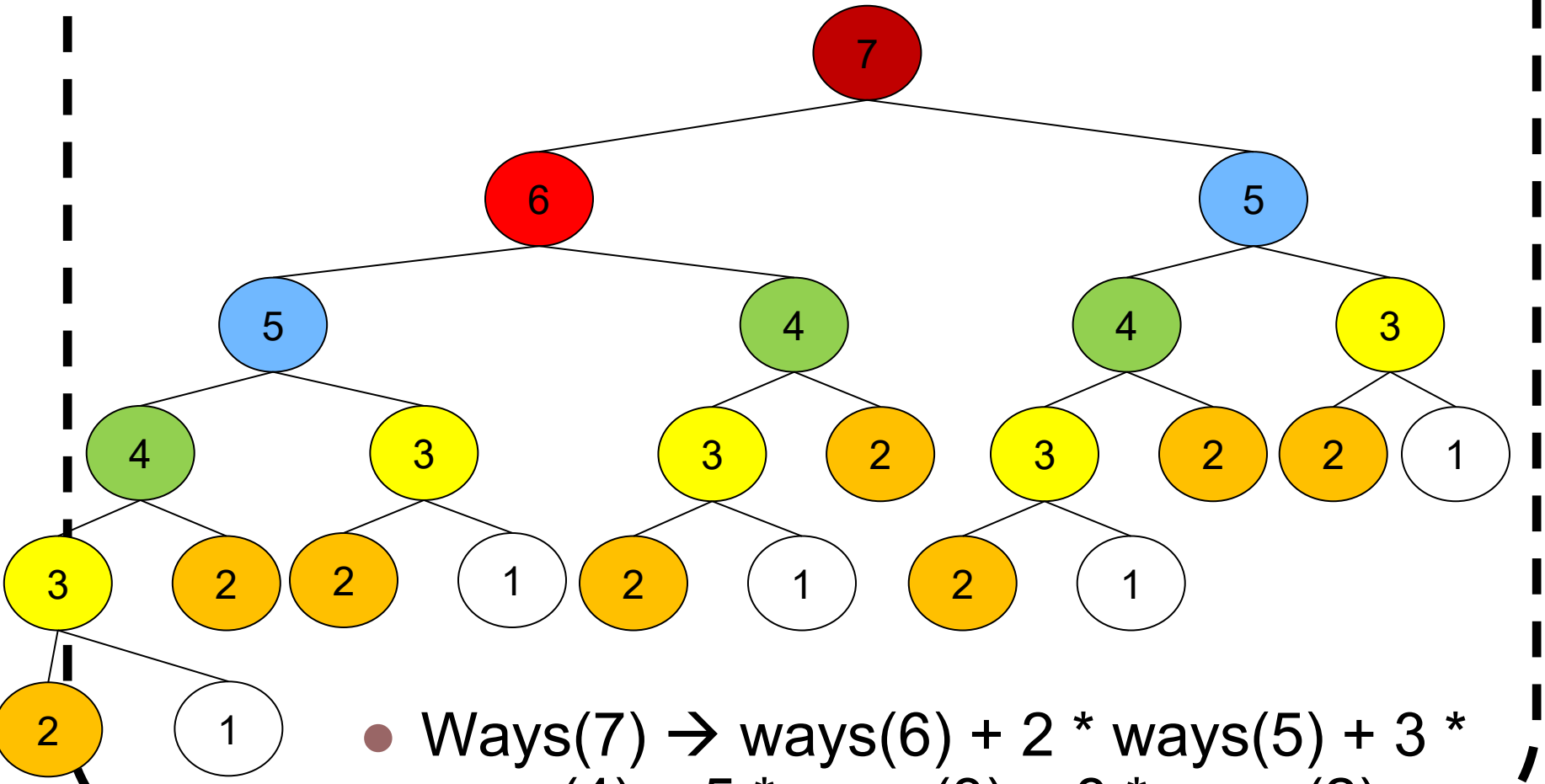
- O soluție optimă a unei probleme conține soluții optime ale subproblemelor (se numește și sub-structură optimală).
- Decompozabilitatea recursivă a problemei P în subprobleme similare P = P_n, P_{n-1}, … P_0 care acceptă soluții din ce în ce mai simple.
- Suprapunerea problemelor (soluția unei probleme P_i participă în procesul de construcție a soluțiilor mai multor probleme P_k de talie mai mare k > i) – memoizare (se foloseşte un tablou pentru salvarea soluţiilor subproblemelor cu scopul de a nu le recalcula).
- În general se folosește o abordare bottom-up, de la subprobleme la probleme.

Exemplu: Urcatul scărilor (3)

- Încercăm abordare top-down:
- ways(i):
  - If i == 0 or i == 1 // ieșirea din recurență
    - RETURN 1
  - RETURN ways(i-1) + ways(i-2)

- Problemă: Are complexitate exponențială – nu este PD !!!!

Exemplu: Urcatul scărilor (4)



- Ways(7) → ways(6) + 2 * ways(5) + 3 * ways(4) + 5 * ways(3) + 8 * ways(2)

Exemplu: Urcatul scărilor (5)

- PD = recursivitate + memoizare (vectorul nw):
- Inițializăm nw[0..n] = -inf // sau orice constantă negativă
- ways(i):
  - IF i == 0 or i == 1 // ieșirea din recurență
    - RETURN 1
  - IF nw[i] == -inf: // nu a fost încă calculată
    - nw[i] = ways(i-1) + ways(i-2) // se calculează o singură dată și se memoizează
  - RETURN nw[i]
- Complexitate Θ(n) – fiecare valoare se calculează o singură dată, și sunt n astfel de valori. (PD top-down corectă)

Problema discretă a rucsacului (1)

- Dându-se n obiecte ce au greutate m_i și valoare v_i, care este valoarea totală maximă pe care o putem transporta într-un rucsac de dimensiune M știind că nu putem fracționa obiectele?

- Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10

- Greedy nu funcționează:
  - Soluția Greedy: m_2 – valoare 19
  - Soluția corectă: $m_1 + m_4$ – valoare 10 + 10 = 20

Problema discretă a rucsacului (2)

- Care este valoarea maximă pentru a transporta n obiecte într-un rucsac de dimensiune M.
- Subproblemă: Care este valoarea maximă pentru a transporta primele i obiecte într-un rucsac de dimensiune j.
- Rezolvare subproblemă (relația de recurență) – se folosește obiectul curent sau nu?

$$sac[i][j] = \begin{cases} \max(sac[i-1][j], v[i] + sac[i-1][j-m[i]]), & dacă\ j \geq m[i] \\ sac[i-1][j], & dacă\ j < m[i] \\ 0, & dacă\ i = 0\ sau\ j = 0 \end{cases}$$

- Soluția se găsește în sac[n][m].

Problema discretă a rucsacului (3)

- Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10

V	m	Ob / M	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	5	1	0	0	0	0	0	10	10	10	10	10	10
19	8	2	0	0	0	0	0	10	10	10	19	19	19
4	4	3	0	0	0	0	4	10	10	10	19	19	19
10	5	4	0	0	0	0	4	10	10	10	19	19	20

- Sac[2][8] = max(sac[1][8], v[2] + sac[1][8-8]) = max(10,19 + 0) = 19

Problema discretă a rucsacului (4)

- Care sunt obiectele selectate?

V	m	Ob / M	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	4	1	0	0	0	0	0	10	10	10	10	10	10
19	8	2	0	0	0	0	0	10	10	10	19	19	19
4	4	3	0	0	0	0	4	10	10	10	19	19	19
10	5	4	0	0	0	0	4	10	10	10	19	19	20

- INIȚIALIZĂM i = n, j = M
- CÂT TIMP i > 0 REPETĂ:
  - DACĂ sac[i][j] > sac[i-1][j] (obiectul i determină schimbarea valorii sac[i][j]): // i = 4
    - Reținem obiectul corespunzător acelei linii // ob_4
    - Din valoarea j scădem greutatea obiectului selectat j = j – m[i] // j = 10 - 5 = 5
  - i = i – 1 // trecem la linia de deasupra

Problema discretă a rucsacului (5)

- Complexitate:
  - Trebuie completată matricea. Câte subprobleme avem?
  - Θ(n * M)
  - Ce complexitate are fiecare pas?
  - Θ(2) – comparăm 2 valori
  - Complexitate finală: Θ(2 * n * M) = Θ(n * M) → complexitate pseudo-polinomială (polinomială în n, dar M nu depinde de n și poate fi oricât de mare)

- Probleme:
  - Pentru valori foarte mari ale lui M, soluția cu programare dinamică nu mai e eficientă
  - Pentru valori reale ale lui m[i], PD nu funcționează (j – m[i] trebuie să fie indice în matrice – număr natural) → problema rucsacului este NP-Hard → nu se poate rezolva eficient polinomial decât dacă obiectele au greutăți numere naturale mici, altfel se rezolvă prin backtracking cu optimizări



Diferențe Greedy – Programare dinamică

Programare lacomă

- Sunt menținute doar soluțiile parțiale curente din care evoluează soluțiile parțiale următoare
- Soluțiile parțiale anterioare sunt eliminate
- Se poate obține o soluție neoptimă. (trebuie demonstrat că se poate aplica).

Programare dinamică

- La construcția unei soluții noi poate contribui orice altă soluție parțială generată anterior
- Se păstrează toate soluțiile parțiale
- Se obține soluția optimă.

Diferenţe divide et impera – programare dinamică

Divide et impera

- abordare top-down – problema este descompusă în subprobleme care sunt rezolvate independent
- putem rezolva aceeaşi problemă de mai multe ori (dezavantaj potenţial foarte mare)

Programare dinamică

- în general abordare bottom-up - se porneşte de la sub-soluţii elementare şi se combină sub-soluţiile mai simple în sub-soluţii mai complicate, pe baza criteriului de optim
- se evită calculul repetat al aceleiaşi subprobleme prin memorarea rezultatelor intermediare (memoizare)

Exemplu: Parantezarea matricilor (Chain Matrix Multiplication)

- Se dă un şir de matrice: $A_1, A_2, ..., A_n$.
- Care este numărul minim de înmulţiri de scalari pentru a calcula produsul:

$$A_1 \times A_2 \times ... \times A_n \ ?$$

- Să se determine una dintre parantezările care minimizează numărul de înmulţiri de scalari.

Înmulţirea matricilor

- A(p, q) x B (q, r) => pqr înmulţiri de scalari.
- Dar înmulţirea matricilor este asociativă (deşi nu este comutativă).
- A(p, q) x B (q, r) x C(r, s)
  (AB)C => pqr + prs înmulţiri
  A(BC) => qrs + pqs înmulţiri
- Ex: p = 5, q = 4, r = 6, s = 2
  (AB)C => 180 înmulţiri
  A(BC) => 88 înmulţiri
- Concluzie: Parantezarea este foarte importantă!

Soluţia banală

- Matrici: $A_1, A_2, ..., A_n$.
- Vector de dimensiuni: $p_0, p_1, p_2, ... , p_n$.
- $A_i(p_{i-1}, p_i) \rightarrow A_1(p_0, p_1), A_2(p_1, p_2), \ldots$
- Dacă folosim căutare exhaustivă şi vrem să construim toate parantezările posibile pentru a determina minimul: $\Omega(4^n / n^{3/2})$.
- Vrem o soluţie polinomială folosind P. D.



Descompunere în subprobleme

- Încercăm să definim subprobleme identice cu problema originală, dar de dimensiune mai mică.

- $\forall\ 1 \le i \le j \le n$:
  - Notăm $A_{i,j} = A_i \times \ldots \times A_j$. $A_{i,j}$ are p_{i-1} linii și p_j coloane: $A_{i,j}(p_{i-1}, p_j)$
  - m[i, j] = numărul optim de înmulţiri pentru a rezolva subproblema $A_{i,j}$
  - s[i, j] = poziţia primei paranteze pentru subproblema $A_{i,j}$
  - Care e parantezarea optimă pentru $A_{i,j}$?

- Problema iniţială: $A_{1,n}$

Combinarea subproblemelor

- Pentru a rezolva $A_{i,j}$
  - Trebuie găsit acel indice $i \leq k < j$ care asigură parantezarea optimă:

$$A_{i,j} = (A_i \times \ldots \times A_k) \times (A_{k+1} \times \ldots \times A_j)$$

$$A_{i,j} = A_{i,k} \times A_{k+1,j}$$

Alegerea optimală

- Căutăm optimul dintre toate variantele posibile de alegere ($i \leq k < j$)
- Pentru aceasta, trebuie însă ca şi subproblemele folosite să aibă soluţie optimală (adică $A_{i, k}$ şi $A_{k+1, j}$ să aibă soluție optimă).

Substructura optimală

- Dacă ştim că alegerea optimală a soluţiei pentru problema $A_{i, j}$ implică folosirea subproblemelor ($A_{i, k}$ şi $A_{k+1, j}$) şi soluţia pentru $A_{i, j}$ este optimală, atunci şi soluţiile subproblemelor $A_{i, k}$ şi $A_{k+1, j}$ trebuie să fie optimale!

- Demonstraţie: Folosind metoda cut-and-paste (metodă standard de demonstrare a substructurii optimale pentru problemele de programare dinamică).

- Observaţie: Nu toate problemele de optim posedă această proprietate! Ex: drumul maxim dintr-un graf aciclic orientat.

Definirea recursivă

- Folosind descompunerea în subprobleme, combinarea subproblemelor, alegerea optimală şi substructura optimală putem să rezolvăm problema prin programare dinamică.
- Următorul pas este să definim recursiv soluţia unei subprobleme.
- Vrem să găsim o formulă recursivă pentru m[i, j] şi s[i, j].



Definirea recursivă (II)

$$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$

- Cazurile de bază sunt m[i, i]
- Noi vrem să calculăm m[1, n]
- Cum alegem s[i, j] ?
- Bottom-up de la cele mai mici subprobleme la cea iniţială.

Rezolvare bottom-up

$m[1,2], m[2,3], m[3,4], \ldots, m[n-3,n-2], m[n-2,n-1], m[n-1,n]$

$m[1,3], m[2,4], m[3,5], \ldots, m[n-3,n-1], m[n-2,n]$

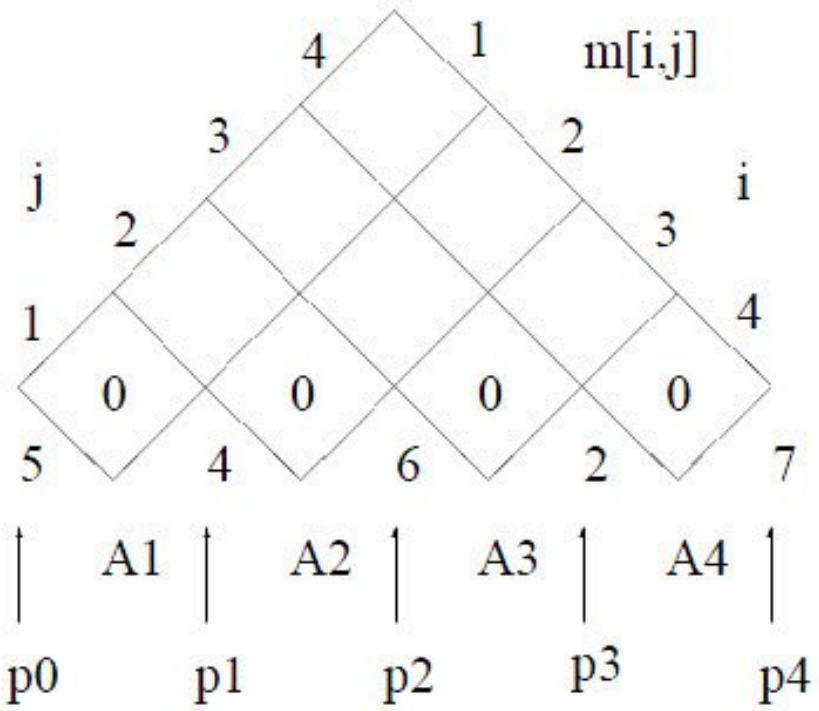
$m[1,4], m[2,5], m[3,6], \ldots, m[n-3,n]$

$\vdots$

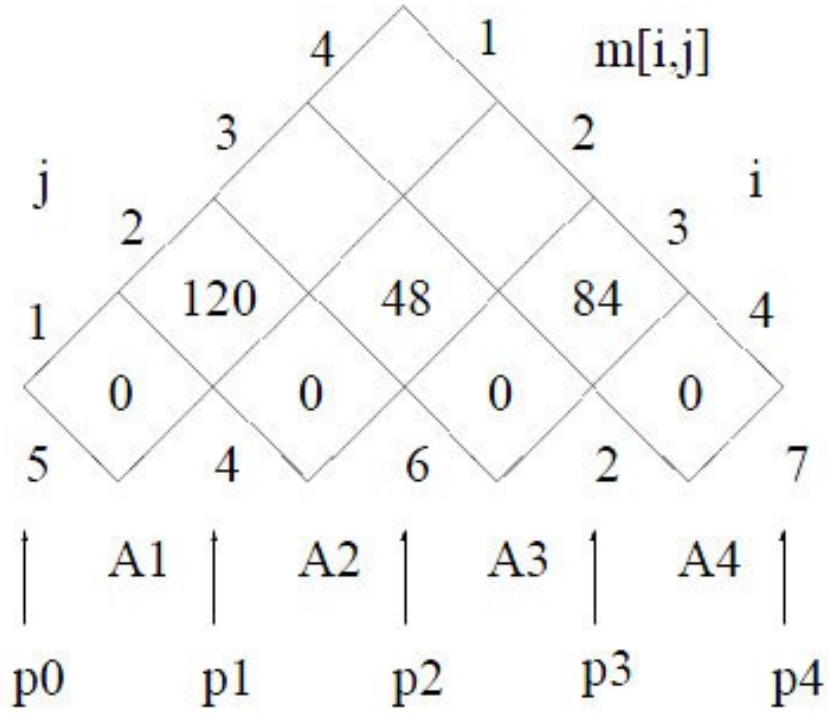
$m[1,n-1], m[2,n]$

$m[1,n]$

Rezolvare - iniţializare



Rezolvare – pas intermediar (I)

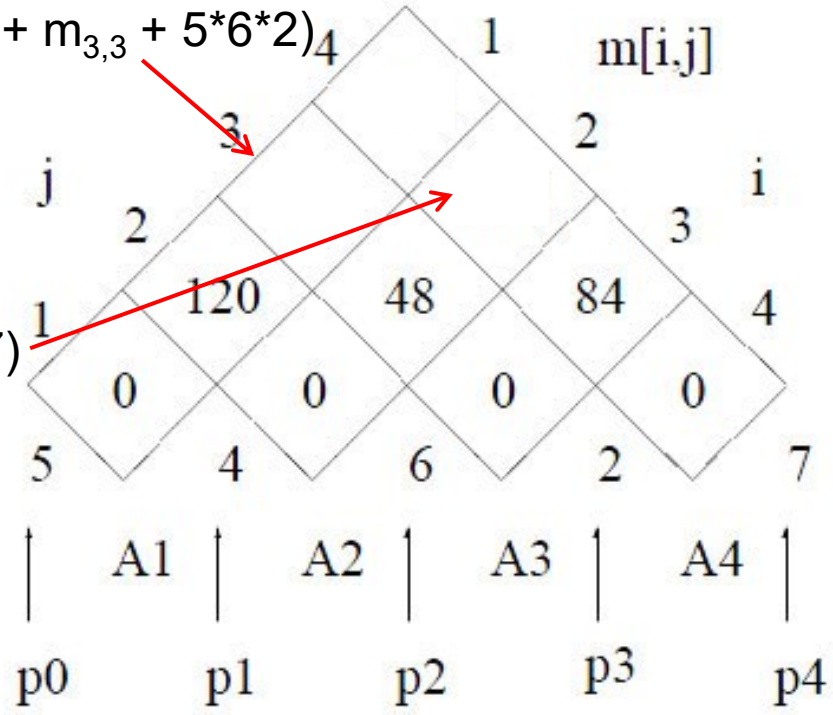


Rezolvare – pas intermediar (II)

$$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$

$A_{1,3} = \{A_1 * A_{2,3}, A_{1,2} * A_3\}$
$m_{1,3} = \min(m_{1,1} + m_{2,3} + 5*4*2, m_{1,2} + m_{3,3} + 5*6*2)$
$s_{1,3} = 1$

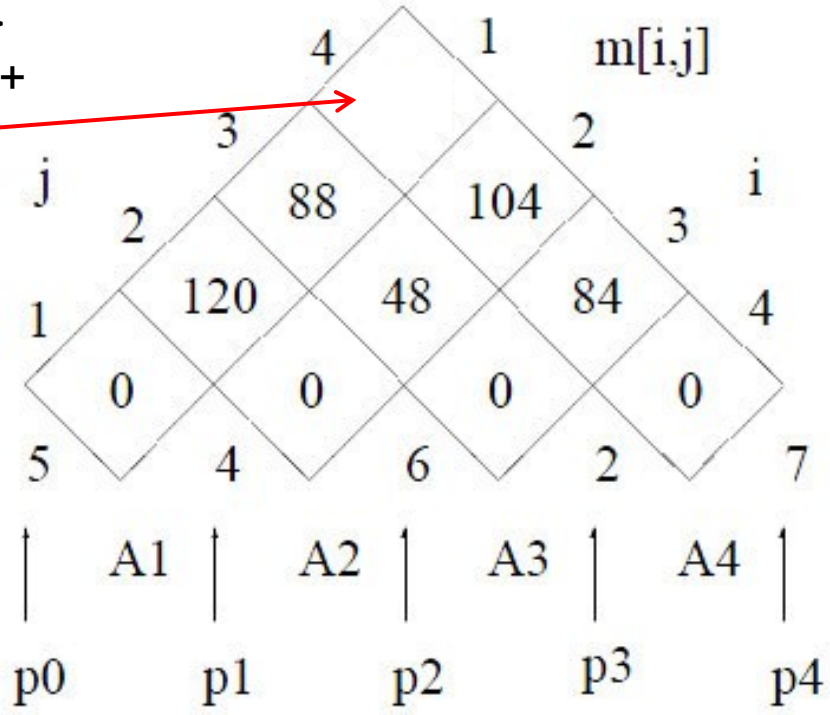
$A_{2,4} = \{A_2 * A_{3,4}, A_{2,3} * A_4\}$
$m_{2,4} = \min(m_{3,4} + 4*6*7, m_{2,3} + 4*2*7)$
$s_{2,4} = 3$



Rezolvare – final

$$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$

$A_{1,4} = \{A_1 * A_{2,4}, A_{1,2} * A_{3,4}, A_{1,3} * A_4\}$
$m_{1,4} = \min(m_{2,4} + 5*4*7, m_{1,2} + m_{3,4} + 5*6*7, m_{1,3} + 5*2*7) = 158$
$s_{1,4} = 3$



Parantezarea optimă este:

$(A_1(A_2A_3))A_4$

Numărul optim de operații

este: 158

Pseudocod

- Înmulțire_matrici (p, n)
  - **Pentru** *i* **de la** 1 **la** *n* // inițializare
    - m[i, i] = s[i, i] = 0
  - **Pentru** *l* **de la** 1 **la** *n - 1* // dimensiune problema
    - **Pentru** *i* **de la 1 la** *n – l* // indice stânga
      - j = i + l // indice dreapta
      - m[i, j] = ∞ // pentru determinare minim
      - **Pentru** *k* **de la** *i* **la** *j – 1*
        - q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j]
        - **Dacă** q < m[i, j]
        - m[i, j] = q
        - s[i, j] = k
  - **Întoarce** *m* și *s*

Complexitate

- Spaţială: $\Theta(n^2)$
  - Pentru memorarea soluţiilor subproblemelor
- Temporală: $O(n^3)$
  - Ns: Număr total de subprobleme: $O(n^2)$
  - Na: Număr total de alegeri la fiecare pas: $O(n)$
  - Complexitatea: $O(n^3)$ este de obicei egală cu Ns x Na



Alt exemplu: Arbori optimi la căutare (AOC)

- Def 2.1: Fie K o mulțime de chei. Un arbore binar cu cheile K este un graf orientat și aciclic A = (V,E) a.î.:
  - Fiecare nod $u \in V$ conține o singură cheie $k(u) \in K$ iar cheile din noduri sunt distincte.
  - Există un nod unic $r \in V$ a.î. i-grad(r) = 0 și $\forall u \neq r$, i-grad(u) = 1.
  - $\forall u \in V$, e-grad(u) ≤ 2; S(u) / D(u) = subarborele stânga / dreapta.

- Def 2.2: Fie K o mulțime de chei peste care există o relație de ordine $\prec$. Un arbore binar de căutare satisface:
  - $\forall u,v,w \in V$ avem $(v \in S(u) \Rightarrow cheie(v) \prec cheie(u)) \wedge (w \in D(u) \Rightarrow cheie(u) \prec cheie(w))$

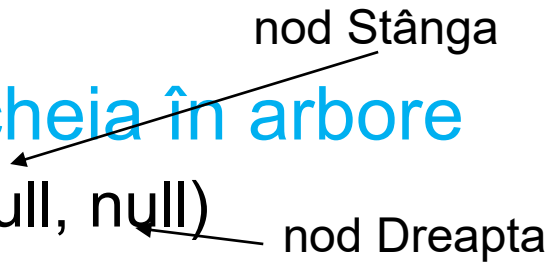
Căutare într-un arbore de căutare

- Caută(elem, Arb)
  - **Dacă** Arb = null
    - **Întoarce** null
  - **Dacă** elem = Arb.val // valoarea din nodul crt.
    - **Întoarce** Arb
  - **Dacă** elem $\prec$ Arb.val
    - **Întoarce** Caută(elem, Arb.st)
  - **Întoarce** Caută(elem, Arb.dr)

Complexitate: $\Theta(\log n)$

Inserţie într-un arbore de căutare

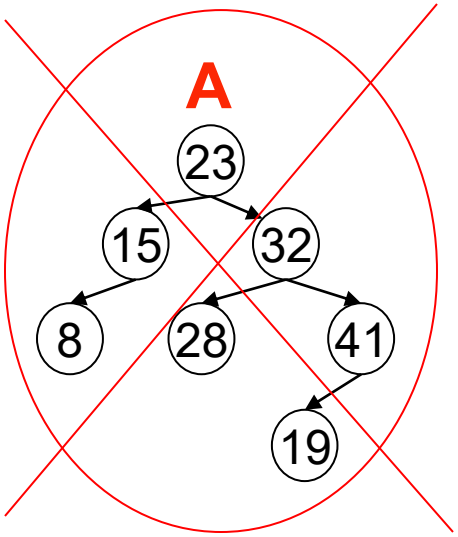
- Inserare(elem, Arb)
  - **Dacă** Arb = vid // adaug cheia în arbore
    - **Întoarce** nod_nou(elem, null, null)
  - **Dacă** elem = Arb.val // valoarea există deja
    - **Întoarce** Arb
  - **Dacă** elem $\prec$ Arb.val
    - **Întoarce** Inserare(elem, Arb.st) // adaugă în stânga
  - **Întoarce** Inserare(elem, Arb.dr) // sau în dreapta

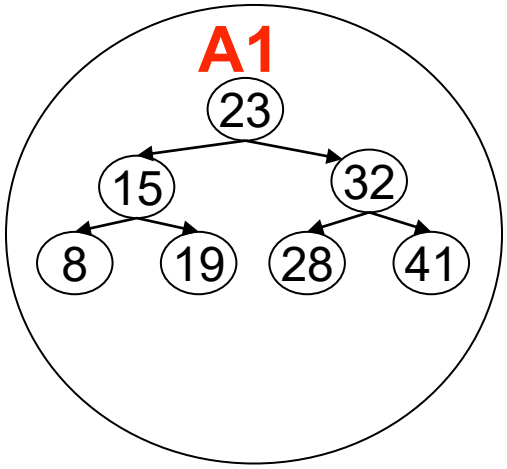


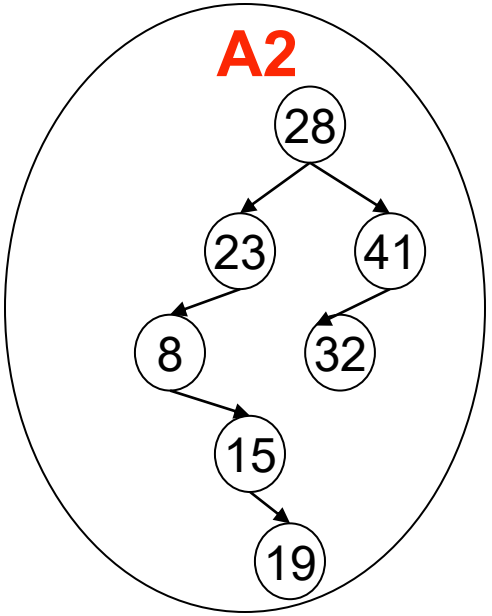
Complexitate: $\Theta(\log n)$

Exemplu de arbori de căutare

- Cu aceleaşi chei se pot construi arbori distincţi

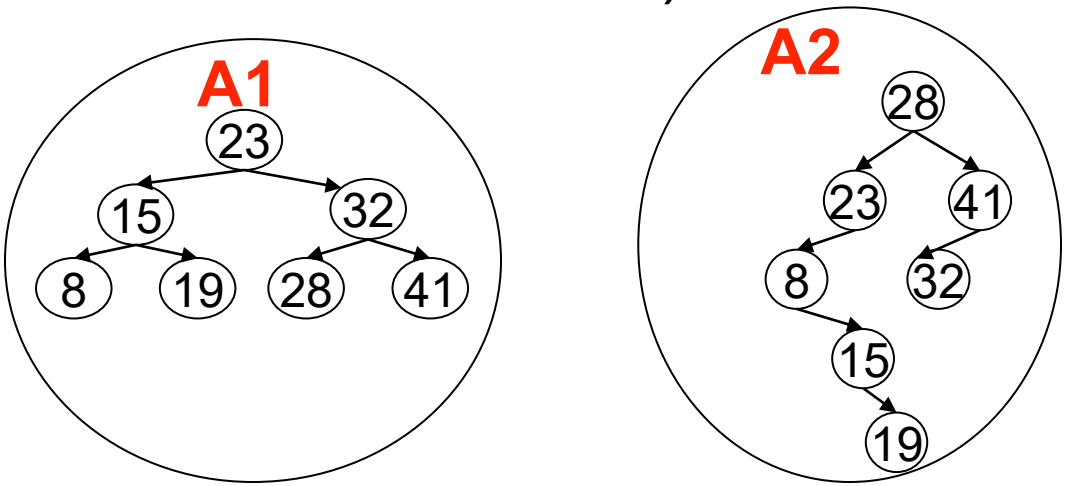






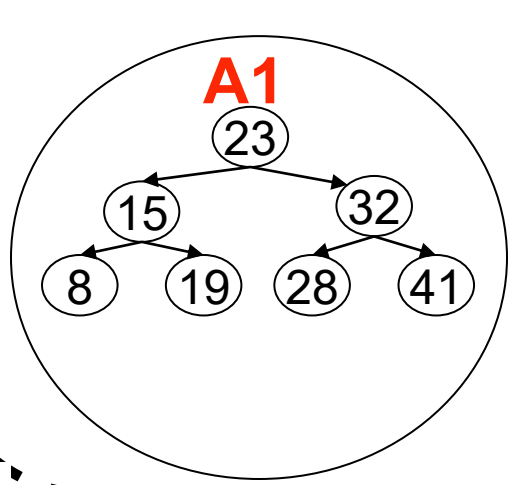
Exemplu (I)

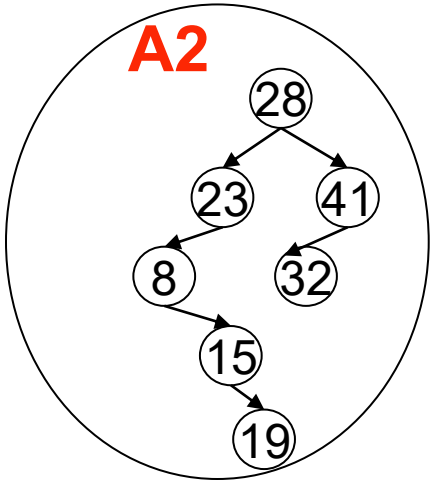
- Presupunem că elementele din A1 şi A2 au probabilităţi de căutare egale:
  - Numărul mediu de comparaţii pentru A1 va fi: (1 + 2 + 2 + 3 + 3 + 3 + 3) / 7 = 2.42
  - Numărul mediu de comparaţii pentru A2 va fi: (1 + 2 + 2 + 3 + 3 + 4 + 5) / 7 = 2.85

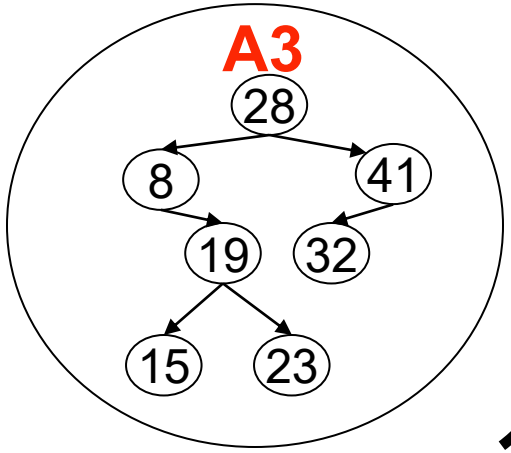


Exemplu (II)

- Presupunem că elementele au următoarele probabilităţi:
  - 8: 0.2; 15: 0.01; 19: 0.1; 23: 0.02; 28: 0.25; 32: 0.2; 41: 0.22;
  - Numărul mediu de comparaţii pentru A1:
    - 0.02*1+0.01*2+0.2*2+0.2*3+0.1*3+0.25*3+0.22*3=2.85
  - Numărul mediu de comparaţii pentru A2:
    - 0.25*1+0.02*2+0.22*2+0.2*3+0.2*3+0.01*4+0.1*5=2.47







Probleme

- Costul căutării depinde de frecvenţa cu care este căutat fiecare termen.
- → Ne dorim ca termenii cei mai des căutaţi să fie cât mai aproape de vârful arborelui pentru a micşora numărul de apeluri recursive.
- Dacă arborele nu este construit prin sosirea aleatorie a cheilor putem ajunge la o simplă listă cu n elemente.
  - Ce se întâmplă când cheile sunt deja sortate crescător / descrescător?

Definiţie AOC

- Definiție: Fie A un arbore binar de căutare cu chei într-o mulțime K; fie $\{x_1, x_2, \ldots x_n\}$ cheile conținute în A, iar $\{y_0, y_1, \ldots y_n\}$ chei reprezentante ale cheilor din K ce nu sunt în A astfel încât: $y_{i-1} \prec x_i \prec y_i, i = \overline{1,n}$. Fie p_i, i = 1,n probabilitatea de a căuta cheia x_i și q_j, j = 0,n probabilitatea de a căuta o cheie reprezentată de y_j. Vom avea relația: $\sum_{i=1}^{n} p_i + \sum_{j=0}^{n} q_j = 1$. A se numește arbore de căutare probabilistică, având costul:

$$Cost(A) = \sum_{i=1}^{n} (nivel(x_i, A) + 1) * p_i + \sum_{j=0}^{n} nivel(y_j, A) * q_j$$

- Definiție: Un arbore de căutare probabilistică având cost minim este un arbore optim la căutare (AOC).

Algoritm AOC naiv

- Generarea permutărilor x_1, ... x_n.
- Construcţia arborilor de căutare corespunzători.
- Calcularea costului pentru fiecare arbore.
- Alegerea arborelui de cost minim.
- Complexitate: $\Omega(n!)$ (deoarece sunt n! permutări).
- → căutăm altă variantă!!!



Construcţia AOC – Notaţii

- $A_{i,j}$ desemnează un AOC cu cheile $\{x_{i+1}, x_{i+2}, \ldots x_j\}$ în noduri și cu cheile $\{y_i, y_{i+1}, \ldots y_j\}$ în frunzele fictive.
- $C_{i,j}$ = Cost $(A_{i,j})$. $Cost(A_{ij}) = \sum_{k=i+1}^{j}(nivel(x_k, A_{ij})+1) * p_k + \sum_{k=i}^{j} nivel(y_k, A_{ij}) * q_k$
- $R_{i,j}$ este indicele α al cheii x_α din rădăcina arborelui $A_{i,j}$.

$$w_{i,j} = \sum_{k=i+1}^{j} p_k + \sum_{k=i}^{j} q_k \quad w_{i,j} = \sum_{k=i+1}^{j-1} p_k + p_j + \sum_{k=i}^{j-1} q_k + q_j = w_{i,j-1} + p_j + q_j$$

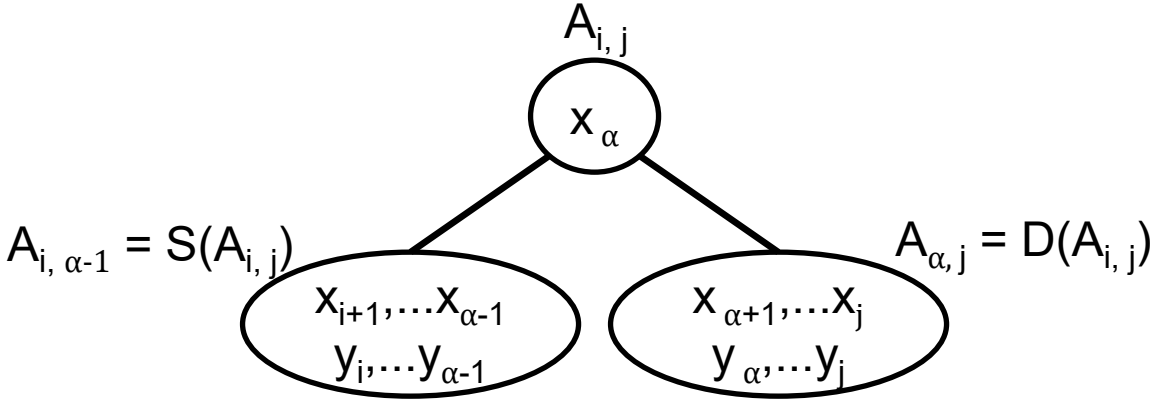
- Observație: $A_{0,n}$ este chiar arborele A, $C_{0,n}$ = Cost (A) iar $w_{0,n} = 1$.

Construcţia AOC - Demonstraţie

- Lemă: Fie $0 \le i < j \le n$ astfel încât există relaţia:

$$C_{i,j} = \min_{i<\alpha\le j}\{C_{i,\alpha-1} + C_{\alpha,j}\} + w_{i,j}$$

și pentru orice α, cu $i < \alpha \le j$, $A_{i,\alpha-1}$ şi $A_{\alpha,j}$ sunt AOC. Atunci arborele de mai jos este AOC ($A_{i,j}$).



Construcţia AOC - Demonstraţie

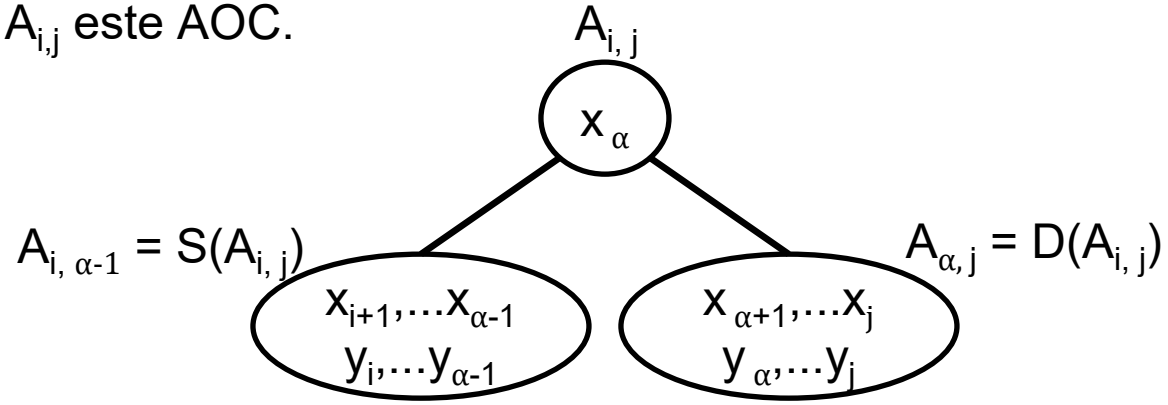
- Demonstrație:

$$Cost(A_{ij}) = \sum_{k=i+1}^{j} (nivel(x_k, A_{ij}) + 1) * p_k + \sum_{k=i}^{j} nivel(y_k, A_{ij}) * q_k$$

$$C_{i,j} = C_{i,\alpha-1} + C_{\alpha,j} + w_{i,j}$$

- $C_{i,j}$ depinde de indicele α al nodului rădăcină.

- Din ipoteză, $A_{i,\alpha-1}$ şi $A_{\alpha,j}$ sunt AOC → $C_{i,\alpha-1}$ şi $C_{\alpha,j}$ sunt minime → $C_{i,j}$ este minim → $A_{i,j}$ este AOC.



Construcţia AOC

- 1. În etapa d, d = 1, 2, … n se calculează costurile și indicele cheilor din rădăcina arborilor AOC $A_{i, i+d}$, i = 0, n-d cu d noduri și d + 1 frunze fictive.

- Arborele $A_{i, i+d}$ conține în noduri cheile $\{x_{i+1}, x_{i+2}, \dots x_{i+d}\}$, iar în frunzele fictive sunt cheile $\{y_i, y_{i+1}, \dots y_{i+d}\}$. Calculul este efectuat pe baza rezultatelor obținute în etapele anterioare.

- Conform lemei avem $$C_{i,i+d} = \min_{i<\alpha\le i+d}\{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

- Rădăcina $A_{i, i+d}$ are indicele $R_{i, j} = \alpha$ care minimizează $C_{i, i+d}$.

- 2. Pentru d = n, $C_{0, n}$ corespunde arborelui AOC $A_{0, n}$ cu cheile $\{x_1, x_2, \dots x_n\}$ în noduri si cheile $\{y_0, y_1, \dots y_n\}$ în frunzele fictive.

Algoritm AOC

AOC(x, p, q, n)

Pentru *i* **de la** 0 **la** *n*

{$C_{i,i} = 0$, $R_{i,i} = 0$, $w_{i,i} = q_i$} // inițializare costuri AOC vid $A_{i,i}$

Pentru *d* **de la** 1 **la** *n*

Pentru *i* **de la** 0 **la** *n-d* // calcul indice rădăcină și cost pentru $A_{i,i+d}$

$j = i + d$, $C_{i,j} = \infty$, $w_{i,j} = w_{i,j-1} + p_j + q_j$

Pentru *α* **de la** *i + 1* **la** *j* // ciclul critic – operații intensive

Dacă ($C_{i,\alpha-1} + C_{\alpha,j} < C_{i,j}$) // cost mai mic?

{ $C_{i,j} = C_{i,\alpha-1} + C_{\alpha,j}$; $R_{i,j} = \alpha$} // update

$C_{i,j} = C_{i,j} + w_{i,j}$ // update

Întoarce gen_AOC(C, R, x, 0, n) // construcție efectivă arbore $A_{0,n}$

// cunoscând indicii

Complexitate???

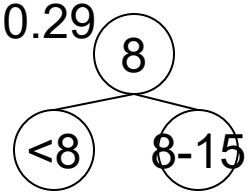
Exemplu constructie AOC (I)

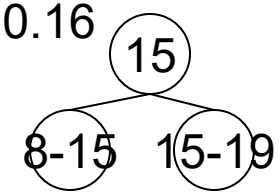
- 8: 0.2; 15: 0.01; 19: 0.1; 23: 0.02; 28: 0.25; (58%)
- [0:8): 0.02; (8:15): 0.07; (15:19): 0.08; (19:23): 0.05; (23:28): 0.05; (28,∞): 0.15 (42%)
- $C_{01}=p_1+q_0+q_1=0.2+0.02+0.07=0.29$
- $C_{12}=p_2+q_1+q_2=0.01+0.07+0.08=0.16$
- $C_{23}=p_3+q_2+q_3=0.1+0.08+0.05=0.23$
- $C_{34}=p_4+q_3+q_4=0.02+0.05+0.05=0.12$
- $C_{45}=p_5+q_4+q_5=0.25+0.05+0.15=0.45$

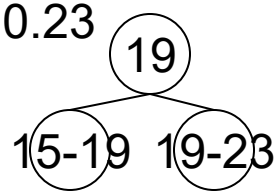
$$w_{i,j}=\sum_{k=i+1}^{j}p_k+\sum_{k=i}^{j}q_k$$

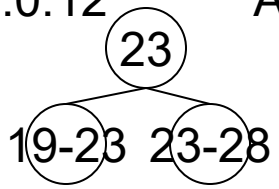
$$w_{i,j}=w_{i,j-1}+p_j+q_j \qquad C_{i,i+d}=\min_{i<\alpha\le i+d}\{C_{i,\alpha-1}+C_{\alpha,i+d}\}+w_{i,i+d}$$

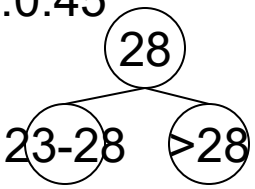
Exemplu constructie AOC (II)

A_{01}:0.29 

A_{12}:0.16 

A_{23}:0.23 

A_{34}:0.12 

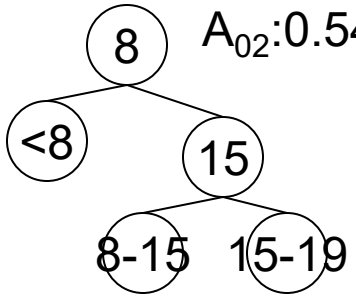
A_{45}:0.45 

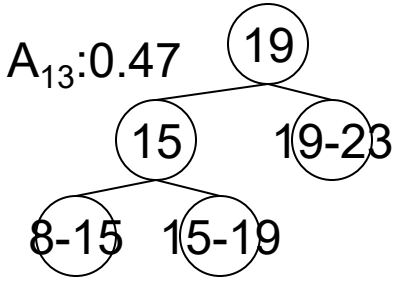
C_{02}= min {(C_{00}+C_{12}),(C_{01}+C_{22})} + w_{02} = min(0.16,0.29) + 0.38 = 0.54 R_{02}= 1 (α=1)

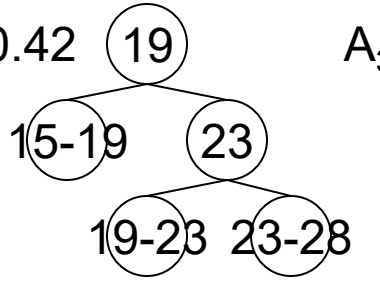
C_{13} = min{C_{11}+C_{23},C_{12}+C_{33}} + w_{13} = min(0.23,0.16) + 0.31 = 0.47 R_{13}= 3 (α=3)

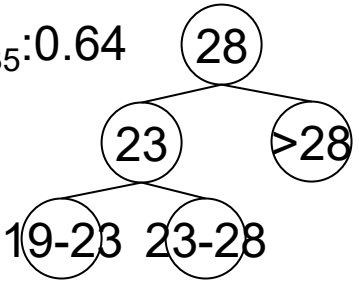
C_{24} = min{C_{34},C_{23}} + w_{24} = min(0.12,0.23) + 0.3 = 0.42 R_{24}= 3

C_{35} = min{C_{45},C_{34}} + w_{35} = min(0.45,0.12) + 0.52 = 0.64 R_{35}= 5

A_{02}:0.54 

A_{13}:0.47 

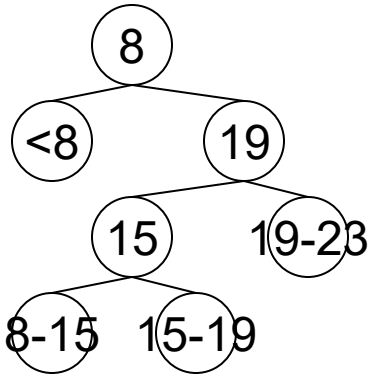
A_{24}:0.42 

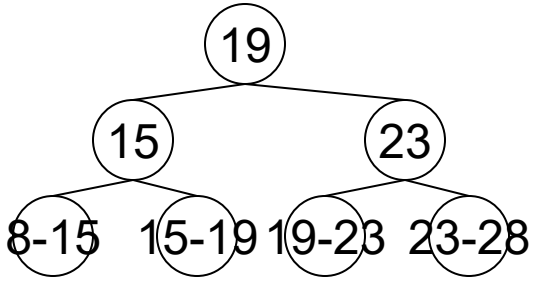
A_{35}:0.64 

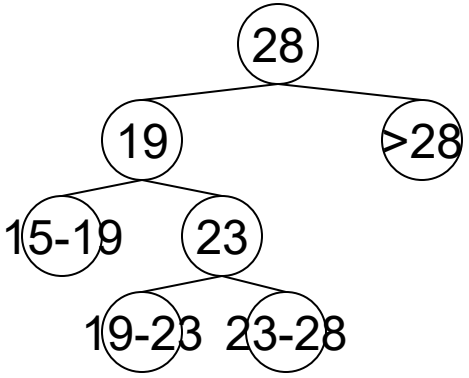
$$w_{i,j} = w_{i,j-1} + p_j + q_j \qquad C_{i,i+d} = \min_{i<\alpha\le i+d}\{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

Exemplu constructie AOC (III)

- $C_{03} = \min(C_{00}+C_{13}, C_{01}+C_{23}, C_{02}+C_{33}) + w_{03} = \min(0.47, 0.52, 0.54) + w_{03} \Rightarrow R_{03} = 1$
- $C_{14} = \min(C_{11}+C_{24}, C_{12}+C_{34}, C_{13}+C_{44}) + w_{14} = \min(0.42, 0.28, 0.47) + w_{14} \Rightarrow R_{14} = 3$
- $C_{25} = \min(C_{22}+C_{35}, C_{23}+C_{45}, C_{24}+C_{55}) + w_{25} = \min(0.64, 0.67, 0.42) + w_{25} \Rightarrow R_{25} = 5$







$$w_{i,j} = \sum_{k=i+1}^{j} p_k + \sum_{k=i}^{j} q_k \qquad C_{i,i+d} = \min_{i<\alpha\le i+d} \{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

AOC – Corectitudine (I)

- Teoremă: Algoritmul AOC construiește un arbore AOC A cu cheile x = $\{x_1, x_2, \ldots x_n\}$ conform probabilităților de căutare p_i, i = 1,n și q_j, j = 0,n.

- Demonstrație: prin inducție după etapa de calcul a costurilor arborilor cu d noduri.

- Caz de bază: d = 0. Costurile $C_{i, i}$ ale arborilor vizi $A_{i, i}$, i = 0, n sunt 0, așa cum sunt inițializate de algoritm.

- Pas de inducție: d ≥ 1. Ip. ind.: pentru orice d' < d, algoritmul AOC calculează costurile $C_{i, i+d'}$ și indicii $R_{i, i+d'}$, ai rădăcinilor unor AOC $A_{i, i+d'}$ cu i = 0, n-d' cu cheile $\{x_{i+1}, x_{i+2}, \ldots x_{i+d'}\}$. Trebuie să arătam că valorile $C_{i, i+d}$ și $R_{i, i+d}$ corespund unor AOC $A_{i, i+d}$ cu i = 0, n-d cu cheile $\{x_{i+1}, x_{i+2}, \ldots x_{i+d}\}$.

AOC – Corectitudine (II)

- Pentru d și i fixate, algoritmul calculează:

$$C_{i,i+d} = \min_{i<\alpha\leq i+d} \{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

- unde costurile $C_{i,\ \alpha-1}$ și $C_{\alpha,\ i+d}$ corespund unor arbori cu un număr de noduri d' = α – 1 – i în cazul $C_{i,\ \alpha-1}$ și d' = i + d – α în cazul $C_{\alpha,\ i+d}$.

- 0 ≤ d' ≤ d – 1 → aceste valori au fost deja calculate în etapele d' < d și conform ipotezei inductive → sunt costuri și indici ai rădăcinilor unor AOC.

- Conform Lemei anterioare, $C_{i,\ j}$ este costul unui AOC. Conform algoritmului → rădăcina acestui arbore are indicele r = $R_{i,\ j}$, iar cheile sunt $\{x_{i+1}, x_{i+2}, \ldots x_{r-1}\}$ $\{x_r\}$ $\{x_{r+1}, x_{r+2}, \ldots x_j\}$ = $\{x_{i+1}, x_{i+2}, \ldots x_j\}$

- Pentru d = n, costul $C_{0,\ n}$ corespunde unui AOC $A_{0,\ n}$ cu cheile x și cu rădăcina de indice $R_{0,\ n}$.

AOC – Concluzii

- Ns: Care este numărul total de subprobleme?

$O(n * (n - d + 1)) = O(n^2)$

- Na: Câte variante de ales avem de făcut pentru determinarea alegerii optimale într-un anumit pas?

$d = j - i = O(n)$ candidaţi pentru rădăcină

- Informal, complexitatea = Ns * Na (Ns = număr subprobleme; Na = număr alegeri)

Complexitate AOC: $O(n^2) * O(n) = O(n^3)$

Optimizare – Knuth: $O(n^2)$

ÎNTREBĂRI?

